

Migration Proposal

Kubernetes Dashboard (Retired) → Headlamp (Active)

Kubermatic Kubernetes Platform (KKP)

Draft v1.0 - March 2026

Tracking Issue: [kubermatic/kubermatic#15287](#)

Scope: kubermatic/kubermatic + kubermatic/dashboard

Status: DRAFT - For Review

Table of Contents

1. Executive Summary
2. Background and Motivation
3. Current Architecture
4. Headlamp Overview
5. Migration Approaches
6. Migration Strategy
7. Detailed Technical Changes
8. Headlamp Plugin Proposal (Phase 2)
9. Challenges and Risks
10. Testing Strategy
11. Timeline and Phases
12. Open Questions
13. References

1. Executive Summary

The Kubernetes Dashboard (v2.7.0), currently integrated into KKP for per-cluster web UI access, has been **retired and archived** by the upstream maintainers. It no longer receives security patches, bug fixes, or feature updates. KKP already marks this feature as deprecated in both the API and frontend UI.

Headlamp (kubernetes-sigs/headlamp) is the actively maintained successor under the Kubernetes SIG umbrella. It provides a modern React/Material UI frontend, a lightweight Go proxy backend, native OIDC support, and a powerful plugin system for extensibility.

This document proposes a **Hybrid migration approach**: Phase 1 performs a direct replacement of the Kubernetes Dashboard deployment with Headlamp (per-cluster, same proxy architecture), followed by Phase 2 which develops KKP-specific Headlamp plugins for deeper integration. The migration spans two repositories: kubermatic/kubermatic (deployment resources) and kubermatic/dashboard (API proxy + Angular frontend).

Two rollout strategies are evaluated: a hard cutover in a single release, and a gradual feature-flag-based transition. The feature-flag approach is recommended for Phase 1 to minimize risk and allow rollback.

2. Background and Motivation

2.1 Kubernetes Dashboard Retirement

The Kubernetes Dashboard project (kubernetes/dashboard) has been moved to kubernetes-retired/dashboard. The project is no longer maintained and will not receive security updates. KKP currently deploys version 2.7.0 (kubernetesui/dashboard:v2.7.0) inside user clusters.

KKP already acknowledges this deprecation. The frontend displays a warning message: *"Kubernetes Dashboard is no longer maintained. This feature is deprecated and may be removed in a future release."*

2.2 Headlamp as Official Successor

Headlamp is a Kubernetes SIG project (kubernetes-sigs/headlamp) that provides a modern, extensible web UI for Kubernetes clusters. It is actively developed, has a growing community, and provides a plugin system for customization. Unlike the retired dashboard, Headlamp receives regular releases with security patches and new features.

2.3 Tracking

GitHub Issue: [kubermatic/kubermatic#15287](https://github.com/kubermatic/kubermatic/issues/15287)

3. Current Architecture

The Kubernetes Dashboard integration in KKP operates across three layers: deployment, API proxy, and frontend. Understanding each layer is essential for planning the migration.

3.1 Deployment Layer (kubermatic/kubermatic)

The Kubernetes Dashboard is deployed **per-user-cluster** in the seed cluster's namespace for that cluster. The deployment is managed by resources in `pkg/resources/kubernetes-dashboard/` in the kubermatic repo.

- **Image:** `kubernetesui/dashboard:v2.7.0`
- **Container port:** 9090 (HTTP, insecure login enabled)
- **Label selector:** `app=kubernetes-dashboard`
- **Replicas:** 2 with anti-affinity
- **Auth:** Uses a mounted kubeconfig secret (`kubernetes-dashboard-kubeconfig`)
- **Flags:** `--enable-insecure-login`
- Health tracked in `ClusterHealth.KubernetesDashboard`
- Enabled by default on new clusters via `ClusterSpec.KubernetesDashboard.Enabled`

3.2 API Proxy Layer (kubermatic/dashboard)

The KKP Dashboard API acts as a reverse proxy to the Kubernetes Dashboard running inside user clusters. This is implemented in 6 Go files under `modules/api/pkg/handler/v2/kubernetes-dashboard/`.

End-to-End Flow:

- **1. Open Dashboard:** User clicks "Open Dashboard" on cluster details page. Frontend constructs URL: `/api/v2/dashboard/login?projectID=...&clusterID=...`
- **2. OIDC Redirect:** API generates nonce, sets secure cookie, redirects to OIDC provider with scopes: `openid, groups, email`
- **3. OIDC Callback:** Provider redirects back with auth code. API exchanges code for tokens, verifies ID token has email claim
- **4. Token Storage:** ID token stored in proxy cookie, user redirected to proxy endpoint
- **5. Proxying:** On each request, API reads token from cookie, opens port-forward to dashboard pod (label: `app=kubernetes-dashboard`, port 9090), creates reverse proxy with `Authorization: Bearer` header

Handler Files:

File	Purpose
<code>handler.go</code>	Base handler interface, <code>isEnabled()</code> global settings check
<code>login.go</code>	OIDC login redirect and callback endpoints
<code>proxy.go</code>	Reverse proxy via port-forward to dashboard pod

File	Purpose
director.go	Strips KKP API prefix, sets Authorization header
request.go	Request types (InitialRequest, OIDCCallbackRequest, ProxyRequest)
response.go	Response types carrying tokens and OIDC state

Route Registration (routes_v2.go:1137-1157): LoginHandler at /dashboard/login with OIDC middleware; ProxyHandler at /projects/{project_id}/clusters/{cluster_id}/dashboard/proxy with token extraction and cluster provider middleware.

3.3 Frontend Layer (Angular)

The Angular frontend provides UI controls for enabling/disabling the dashboard, displaying its health status, and launching it from the cluster details page.

Component / File	Purpose
shared/entity/settings.ts	enableDashboard boolean in AdminSettings (defaults true)
shared/entity/cluster.ts	KubernetesDashboard class with enabled? field
shared/entity/health.ts	kubernetesDashboard health state tracking
cluster/details/cluster/component.ts	"Open Dashboard" button, health check, proxy URL
cluster/details/cluster/edit-cluster/	KubernetesDashboardEnabled form control
wizard/step/cluster/component.ts	Dashboard toggle in cluster creation wizard
settings/admin/defaults/	Global enable/disable toggle (requires OIDC feature gates)
core/services/cluster.ts	getDashboardProxyURL() method
shared/constants/common.ts	KUBERNETES_DASHBOARD_DEPRECATED_MESSAGE constant

4. Headlamp Overview

4.1 Architecture

Headlamp consists of two main components: a **Go proxy backend** (headlamp-server) and a **React/Material UI frontend** SPA. The server handles kubeconfig injection, authentication, static file serving, and WebSocket proxying for streaming operations.

4.2 Key Features

- **Modern stack:** React + Material UI (MUI), TypeScript, actively maintained
- **Multi-cluster support:** Native support for managing multiple clusters
- **Plugin system:** Extensible via JavaScript/TypeScript plugins loaded at runtime
- **OIDC authentication:** Built-in OIDC support with configurable scopes
- **Token-based auth:** ServiceAccount tokens, bearer tokens via kubeconfig
- **Desktop app:** Available as an Electron-based desktop application
- **Helm chart:** Official Helm chart for in-cluster deployment
- **RBAC visualization:** Enhanced view of cluster roles and bindings

4.3 Plugin System

Headlamp's plugin system allows extending the UI without forking the project. Plugins are npm packages using the @kinvolk/headlamp-plugin SDK. They can register new sidebar entries, detail view panels, actions, and custom pages. Plugins are distributed as JavaScript bundles loaded from a plugins directory at runtime.

4.4 Comparison with Kubernetes Dashboard

Aspect	K8s Dashboard (Retired)	Headlamp (Active)
Status	Archived, unmaintained	Active SIG project
Frontend	Angular	React + MUI
Backend	Go	Go (proxy-based)
Multi-cluster	Single cluster only	Native multi-cluster
Extensibility	No plugin system	Rich plugin API
Desktop app	None	Electron desktop app
Auth	Token/kubeconfig	OIDC, tokens, kubeconfig
RBAC	Basic view	Enhanced visualization

5. Migration Approaches

5.1 Approach A: Direct Drop-in Replacement

Replace the Kubernetes Dashboard deployment with Headlamp in the kubermatic repo. Adjust the KKP API proxy handler for Headlamp's port, labels, and path structure. Minimal frontend changes (rename labels, update health checks, remove deprecation warnings).

- **Pro:** Fastest path to production, minimal architectural change
- **Con:** Does not leverage Headlamp's plugin system or multi-cluster capabilities
- **Effort:** Medium (3-4 weeks)

5.2 Approach B: Full KKP-Integrated Headlamp

Build custom Headlamp plugins from day one: KKP project context, cloud provider info, MLA integration, cluster updates. Deploy a single shared Headlamp instance per seed with multi-cluster support instead of per-cluster deployments.

- **Pro:** Best user experience, resource efficient, leverages full Headlamp capabilities
- **Con:** Large upfront effort, major architectural change, complex auth delegation
- **Effort:** High (8-12 weeks)

5.3 Approach C: Hybrid (Recommended)

Start with Approach A (per-cluster Headlamp replacement) to quickly restore a working, maintained dashboard. Then incrementally develop KKP-specific Headlamp plugins in Phase 2. This provides fast time-to-value while opening the door for deeper integration.

- **Pro:** Fast initial delivery + clear extensibility roadmap
- **Con:** Two phases of work, initial deployment without KKP-specific features
- **Effort:** Medium initially (3-4 weeks), then incremental (4-6 weeks for plugins)

5.4 Comparison

Criteria	A: Drop-in	B: Full Integration	C: Hybrid
Time to production	3-4 weeks	8-12 weeks	3-4 weeks (Phase 1)
Risk	Low	High	Low
User impact	Minimal change	Major UX improvement	Incremental improvement
Extensibility	Limited	Full	Full (after Phase 2)
Architecture change	Minimal	Major	Minimal then incremental
Rollback complexity	Simple	Complex	Simple

6. Migration Strategy

Two rollout strategies are evaluated. The team should choose based on release timeline constraints and risk appetite.

6.1 Option 1: Hard Cutover

A single KKP release replaces Kubernetes Dashboard with Headlamp. Existing clusters with `kubernetesDashboard.enabled: true` are automatically migrated to Headlamp on KKP upgrade. The old Kubernetes Dashboard pods are cleaned up by the cluster controller.

- **Pro:** Simpler implementation, no dual-path code, clean codebase
- **Con:** No rollback without KKP downgrade, higher risk, all-or-nothing for all clusters
- **When to choose:** Confidence in Headlamp compatibility is high, testing is thorough

6.2 Option 2: Gradual with Feature Flag (Recommended)

Introduce a feature gate (e.g., `HeadlampDashboard`) alongside the existing `EnableDashboard` setting. Both dashboards are deployable; admins choose which one is active. This allows opt-in testing before full rollout.

Implementation options for the feature flag:

- **Option A:** New `dashboardType` field in `AdminSettings`: `'kubernetes-dashboard' | 'headlamp'`
- **Option B:** Separate `enableHeadlamp` boolean alongside `enableDashboard`
- **Option C:** KKP feature gate in `FeatureGates` configuration

Recommended rollout: Feature flag in release N (default: Kubernetes Dashboard). Switch default to Headlamp in release N+1. Remove Kubernetes Dashboard code in release N+2.

Criteria	Hard Cutover	Feature Flag
Implementation complexity	Lower	Higher (dual paths)
Risk	Higher	Lower (rollback possible)
Testing burden	Higher (must be thorough pre-release)	Lower (progressive rollout)
Code cleanliness	Clean from day 1	Temporary dual-path code
User disruption	All at once	Gradual, opt-in

7. Detailed Technical Changes

7.1 KKP Core Repo (kubermatic/kubermatic)

7.1.1 Deployment Manifests

Replace pkg/resources/kubernetes-dashboard/deployment.go:

- Change container image from kubernetesui/dashboard:v2.7.0 to ghcr.io/headlamp-k8s/headlamp:<version> (pin specific version)
- Update container args: Headlamp uses -in-cluster and -plugins-dir flags
- Update container port (Headlamp defaults to port 4466)
- Update label selector from app=kubernetes-dashboard to app=headlamp
- Review volume mounts: Headlamp may use in-cluster ServiceAccount rather than a mounted kubeconfig
- Update liveness/readiness probes (Headlamp serves on /)

7.1.2 RBAC Resources

Review and update ClusterRole/ClusterRoleBinding for Headlamp. Headlamp's server needs read access to cluster resources. If plugins need write access, scope the RBAC accordingly. Consider whether the existing RBAC setup for Kubernetes Dashboard is sufficient or needs expansion.

7.1.3 Health Checks

Update the health check controller that monitors dashboard pod status. Change the label selector and potentially the health endpoint path. The health status field in ClusterHealth should be updated or aliased.

7.1.4 Deletion Resources

Update deletion.go to clean up Headlamp resources instead of Kubernetes Dashboard resources when a cluster is deleted.

7.2 Dashboard Repo - Go API

7.2.1 Proxy Handler

modules/api/pkg/handler/v2/kubernetes-dashboard/proxy.go

- Update ContainerPort reference: Headlamp uses port 4466 by default (vs 9090 for K8s Dashboard)
- Update AppLabel reference: change from app=kubernetes-dashboard to app=headlamp
- Update CSP header: Headlamp's React app may require script-src 'self' in addition to style-src
- The port-forward approach remains the same: open port-forward to Headlamp pod, reverse proxy requests

- Consider caching port-forwards (current code opens a new one per request with a TODO comment)

7.2.2 Login Flow

`modules/api/pkg/handler/v2/kubernetes-dashboard/login.go`

The OIDC login flow (redirect -> exchange -> cookie store -> proxy) should remain largely unchanged. Headlamp supports OIDC natively, but KKP's proxy model means KKP mediates auth. Key verification: confirm Headlamp accepts the ID token as a Bearer token for API calls.

7.2.3 Director (Path Rewriting)

`modules/api/pkg/handler/v2/kubernetes-dashboard/director.go`

The `getBasePath()` function strips everything before "proxy" in the URL. This should work unchanged if the route prefix stays the same. Verify that Headlamp's client-side routing (React Router) is compatible with this path stripping approach. Headlamp may need a `-base-url` configuration to handle the proxied path prefix correctly.

7.2.4 Routes

`modules/api/pkg/handler/v2/routes_v2.go:1137-1157`

- Consider renaming route from `/dashboard/` to `/headlamp/` or keeping generic name
- If feature-flag approach: register both route sets, dispatch based on admin setting
- Package may be renamed from `kubernetesdashboard` to `headlamp`

7.2.5 API Types

`modules/api/pkg/api/v1/types.go`

- `KubernetesDashboard` struct (line 1015): rename to `Headlamp` or `ClusterDashboard`, or add new field alongside
- `ClusterHealth.KubernetesDashboard` (line 1462): update field name
- Swagger/OpenAPI spec will need regeneration

7.2.6 Cluster Defaults

`modules/api/pkg/resources/cluster/cluster.go:44-69` - Update defaulting logic to set Headlamp enabled by default instead of Kubernetes Dashboard.

7.3 Dashboard Repo - Angular Frontend

File / Area	Change Required
<code>shared/entity/cluster.ts</code>	Rename <code>KubernetesDashboard</code> class to <code>Headlamp</code> (or add new, deprecate old)
<code>shared/entity/health.ts</code>	Update <code>HealthStatusKey.KubernetesDashboard</code> enum value
<code>shared/entity/settings.ts</code>	Rename <code>enableDashboard</code> to <code>enableHeadlamp</code> or add alongside

File / Area	Change Required
cluster/details/cluster/component.ts	Update isKubernetesDashboardHealthy getter, getProxyURL(), remove deprecation warning
cluster/details/cluster/template.html	Rename button text, update CSS classes, remove deprecation icon
cluster/details/cluster/edit-cluster/	Update KubernetesDashboardEnabled form control and spec patch
wizard/step/cluster/component.ts	Update KubernetesDashboardEnabled control and spec generation
settings/admin/defaults/	Update label from "Enable Kubernetes Dashboard" to "Enable Headlamp"
core/services/cluster.ts	Update getDashboardProxyURL() method name and path
shared/constants/common.ts	Remove KUBERNETES_DASHBOARD_DEPRECATED_MESSAGE
Cypress E2E tests	Update selectors, page objects, and fixture data
Swagger client models	Regenerate from updated swagger.json

8. Headlamp Plugin Proposal (Phase 2)

After the initial replacement (Phase 1), KKP-specific Headlamp plugins can provide deeper integration and a better user experience. Plugins are JavaScript bundles loaded at runtime from a plugins directory mounted into the Headlamp container.

8.1 Proposed Plugins

Plugin	Description	Priority
KKP Project Context	Sidebar entry showing KKP project info, breadcrumb integration linking back to KKP Dashboard, project-scoped resource filtering	High
Cloud Provider Info	Display cloud provider details (AWS/GCP/Azure/etc.) for the cluster, show datacenter/region info, link to provider console	Medium
MLA Integration	Direct links to Grafana/Loki dashboards from Headlamp, embedded monitoring panels, alert status display	Medium
Cluster Updates	Show available Kubernetes version upgrades, link back to KKP Dashboard for upgrade action, display update windows	Low
Enhanced RBAC	Show KKP-managed roles and bindings, visualize project member permissions, display group-project bindings (EE)	Low

8.2 Plugin Architecture

- **Development:** Use @kinvolk/headlamp-plugin npm SDK
- **Registration:** Plugins call `registerRoute()`, `registerSidebarEntry()`, `registerDetailsViewHeaderAction()`
- **KKP API access:** Plugins call KKP API endpoints via the Headlamp proxy or direct fetch (requires CORS or proxy configuration)
- **Distribution:** Bake plugins into the Headlamp container image as part of the KKP build, or mount via ConfigMap/PersistentVolume

8.3 Plugin Distribution Strategy

Recommended: Bake plugins into a custom Headlamp container image built as part of the KKP release pipeline. This ensures version compatibility between plugins and the Headlamp version. The Dockerfile would copy plugin bundles into `/headlamp/plugins/` during build.

9. Challenges and Risks

Risk	Severity	Mitigation
Port-forward scalability: current proxy opens a new port-forward per request	Medium	Implement port-forward caching/pooling (existing TODO in proxy.go). This is a pre-existing issue not introduced by migration.
CSP header differences: Headlamp's React SPA may need different Content-Security-Policy	Low	Test Headlamp behind the proxy, determine required CSP directives, update header in proxy handler.
Path rewriting compatibility: Headlamp uses React Router (client-side routing)	Medium	Configure Headlamp's -base-url flag to match the proxy path prefix. Test navigation within the proxied Headlamp.
OIDC token compatibility: verify Headlamp accepts KKP-issued ID tokens	High	Prototype early: deploy Headlamp in a test cluster, verify the existing OIDC flow works with Headlamp's token acceptance.
Resource overhead: Headlamp may have different CPU/memory profile per cluster	Low	Benchmark Headlamp resource usage, adjust deployment resource requests/limits accordingly.
Backward compatibility: existing clusters with <code>kubernetesDashboard.enabled: true</code>	Medium	Feature flag approach allows gradual migration. Cluster controller handles cleanup of old dashboard resources.
Plugin API stability: Headlamp plugin API is still evolving	Medium	Pin to stable Headlamp version for Phase 1. Phase 2 plugins should target a specific plugin API version.

10. Testing Strategy

10.1 Unit Tests

Go handler tests for the updated proxy logic using mock port-forwarder. Verify correct headers, path rewriting, and token handling. Update existing tests in `modules/api/pkg/handler/v2/kubernetes-dashboard/`.

10.2 Integration Tests

Deploy Headlamp in a test seed cluster namespace. Verify the full proxy flow: OIDC login -> token exchange -> proxy -> Headlamp UI loads correctly. Test with real OIDC provider.

10.3 E2E Tests (Cypress)

Update Cypress tests for: admin settings toggle, cluster creation wizard checkbox, cluster detail page "Open Dashboard" button visibility and health status display. Update page objects and selectors in the Cypress test suite.

10.4 Upgrade / Migration Tests

Critical: verify existing clusters with `kubernetesDashboard.enabled: true` are correctly migrated to Headlamp on KKP upgrade. Verify old Kubernetes Dashboard resources are cleaned up. Test both fresh cluster creation and upgrade scenarios.

10.5 Performance Tests

Measure proxy latency overhead for Headlamp vs Kubernetes Dashboard baseline. Benchmark Headlamp pod startup time, memory usage, and CPU consumption per cluster.

11. Timeline and Phases

Phase	Duration	Description	Deliverables
Phase 0: Investigation	1-2 weeks	Deploy Headlamp manually in a test cluster. Verify proxy compatibility, CSP requirements, path rewriting, OIDC token acceptance.	Compatibility report, prototype branch
Phase 1: Direct Replacement	3-4 weeks	Replace K8s Dashboard deployment with Headlamp in kubermatic repo. Update API proxy handlers and frontend UI. Implement feature flag.	PRs for both repos, updated tests, release notes
Phase 2: KKP Plugins	4-6 weeks	Develop KKP-specific Headlamp plugins: project context, provider info, MLA integration.	Plugin packages, custom Headlamp image, documentation

Phase	Duration	Description	Deliverables
Phase 3: Cleanup	1 week	Remove K8s Dashboard code paths, remove feature flag, update documentation.	Clean codebase, updated docs

Total estimated effort: 9-13 weeks across all phases. Phase 1 can be shipped independently. Phase 2 plugins can be delivered incrementally across multiple releases.

12. Open Questions

1. **API route naming:** Should the route path change from `/dashboard/` to `/headlamp/`, or keep a generic name like `/cluster-ui/`?
2. **CRD field naming:** Should `KubernetesDashboard` in the CRD/API types be renamed to `ClusterDashboard` (generic) or `Headlamp` (specific)?
3. **OIDC mediation:** Does Headlamp's native OIDC support obviate the need for KKP's OIDC login flow, or should KKP continue to mediate authentication?
4. **Version pinning:** What Headlamp version should KKP target? Should KKP track Headlamp releases or pin a specific version per KKP release?
5. **Plugin distribution:** Should KKP plugins be baked into the container image (simpler) or loaded dynamically via `ConfigMap` (more flexible)?
6. **Deployment model:** Is the per-cluster deployment model still the right choice, or should a single Headlamp instance serve multiple clusters per seed?
7. **RBAC scope:** What are the RBAC implications of Headlamp's built-in capabilities (it can create/delete resources by default)?

13. References

Kubernetes Dashboard (Retired):

<https://github.com/kubernetes-retired/dashboard>

Headlamp Project: <https://github.com/kubernetes-sigs/headlamp>

Headlamp Documentation: <https://headlamp.dev/docs/latest/>

Headlamp Plugin Development:

<https://headlamp.dev/docs/latest/development/plugins/>

Headlamp Helm Chart: <https://artifacthub.io/packages/helm/headlamp/headlamp>

KKP Documentation: <https://docs.kubermatic.com/kubermatic/>

Tracking Issue: <https://github.com/kubermatic/kubermatic/issues/15287>